

React Native

Learning Field Guide

Gautam's Bug Bible + Prevention Laws

Windows · Expo SDK 54 · Android

Every bug hit face-first during 2 days of building the BeReal clone.
Each one documented with its type, root cause, exact fix, and prevention rule.
Read this alongside Part 2 (Code Walkthrough) for full context.

- #01 [HIGH] Property 'isAuth' does not exist on type 'AuthContextType'
- #02 [HIGH] signup.tsx auto-navigates to onboarding on mount
- #03 [HIGH] Attempted to navigate before mounting Root Layout
- #04 [HIGH] VS Code opened in wrong folder — all imports broken
- #05 [HIGH] GestureHandlerRootView missing — crash on gesture
- #06 [MEDIUM] SafeAreaView edges prop TypeScript error on Android
- #07 [HIGH] Android permission popup never appeared
- #08 [HIGH] Supabase storage bucket policy mess
- #09 [MEDIUM] Typo: 'username' instead of 'username' in query

What's In Part 3

This is the debugging companion to Part 2. Every bug is documented with 4 things: what type of error it was, what the root cause was, the exact fix, and how to prevent it next time. Followed by the 10 prevention laws and a complete quick reference cheatsheet.

Errors #01-09

9 bugs with root cause analysis

Laws #01-10

The 10 prevention laws

Cheatsheet

Print this. Stick it on your monitor.

Closing

What comes next for EarGuard

Error #01

Source: app/_layout.tsx

Property 'isAuth' does not exist on type 'AuthContextType'**HIGH****What is this error?**

TypeScript threw a red underline in VS Code on `const { isAuth } = useAuth()`. The message: 'Property isAuth does not exist on type AuthContextType'. The app would not compile.

Type

TypeScript compile-time error. Red underline in VS Code. App refuses to build until fixed.

Root Cause

AuthContextType defines exactly 3 properties: user, signUp, signIn. No isAuth anywhere. You tried to destructure a property that does not exist in the interface. TypeScript caught this before the code ever ran on your device.

Fix

Change: `const { isAuth } = useAuth()` To: `const { user } = useAuth()` Then change: `if (!isAuth)` To: `if (!user)`. The property name must match exactly what is defined in AuthContextType.

How to Prevent This Next Time

After writing any `useAuth()` destructure, hover over the returned object in VS Code. TypeScript autocomplete shows you the exact available properties. Never guess property names. Let the interface tell you what exists.

Error #02

Source: app/(auth)/signup.tsx

signup.tsx auto-navigates to onboarding on mount**HIGH****What is this error?**

Every time the signup screen opened, the app immediately jumped to the onboarding screen before the user typed anything. The signup form was never usable.

Type

Silent logic bug. No crash, no error message. Wrong screen appeared before user interaction.

Root Cause

`useEffect(() => { router.push('/(auth)/onboarding'); }, [])` was placed in `signup.tsx`. The empty `[]` dependency array means 'run once on mount'. Mount = the instant the signup screen appears. So navigation fired before the user interacted with anything. This `useEffect` is STILL in your uploaded `signup.tsx` file.

Fix

Delete the entire `useEffect` block from `signup.tsx`. Move `router.replace('/(auth)/onboarding')` into the `try` block inside `handleSignUp`, AFTER `await signUp()` completes successfully. Navigation should happen after user action — not on screen mount.

How to Prevent This Next Time

`useEffect([])` = setup code that runs once on mount. Navigation triggered by a user action belongs INSIDE the button handler, not in `useEffect`. Rule: if you find yourself putting `router.push` inside `useEffect([])`, stop and reconsider.

```
// The correct pattern
// WRONG: navigation in useEffect
useEffect(() => {
  router.push("/(auth)/onboarding"); // fires on MOUNT
}, []);
// RIGHT: navigation inside button handler
const handleSignUp = async () => {
  try {
    await signUp(email, password);
    router.replace("/(auth)/onboarding"); // fires after success
  } catch (error: any) { ... }
};
```

Error #03

Source: app/_layout.tsx

Attempted to navigate before mounting Root Layout**HIGH****What is this error?**

The app crashed with a red error screen on Android immediately on launch. The error message: 'Attempted to navigate before mounting the Root Layout component. Ensure the Root Layout component is rendering a Navigator.'

Type

Runtime crash. Red error screen on Android. App unusable.

Root Cause

Navigation calls (`router.replace`) were happening synchronously during the render phase — before Expo Router had finished mounting its navigation tree. Expo Router needs a small window of time after the component renders to fully initialise its navigator. Calling navigation methods before that window closes = crash. Android is more sensitive to this timing issue than iOS.

Fix

Two changes in `_layout.tsx`: (1) Add guard: `if (!navigationState?.key) return;` at top of `useEffect`. This stops execution if Expo Router is not ready yet. (2) Wrap the `router.replace` call in `setTimeout(fn, 0)`. This defers the navigation to the next event loop tick, after the navigator is mounted.

How to Prevent This Next Time

Never call `router.push` or `router.replace` synchronously during a render or directly in `useEffect` without a `navigationState` ready-check. Always pair the ready-check guard with `setTimeout(fn, 0)` for initial navigation.

```
// The fix – ready-check + deferred navigation
useEffect(() => {
  if (!navigationState?.key) return;
  // navigationState?.key = optional chaining.
  // If navigationState is null -> returns undefined -> stops here.
  // .key existing = Expo Router navigation tree is fully mounted.
  const timeout = setTimeout(() => {
    if (!user) router.replace("/(auth)/login");
    else router.replace("/(tabs)");
  }, 0);
  // setTimeout(fn, 0) = defer to next event loop tick.
  // The current render finishes first, THEN navigation fires.
  return () => clearTimeout(timeout);
}, []);
```

Error #04

Source: Development environment

VS Code opened in wrong folder — all imports broken

HIGH

What is this error?

Every import in every file showed a red 'Cannot find module' error in VS Code, even for packages that were correctly installed. Files created during the session ended up in the wrong directory.

Type

Environment configuration error. Not a code bug. VS Code TypeScript server pointing at the wrong directory.

Root Cause

VS Code was opened in the parent React_Native folder instead of the berealclone project folder. npm packages are installed per-project inside node_modules inside the project folder. When VS Code opens the parent folder, its TypeScript server looks for node_modules in the parent — which does not exist. All imports appear broken even though packages are correctly installed inside berealclone.

Fix

Close VS Code completely. Open PowerShell. cd into berealclone. Verify with pwd that you are in the right place. Run: code . (the dot means open CURRENT directory). Also move any files created in the wrong folder with Move-Item.

How to Prevent This Next Time

Run `pwd` before every coding session. Always open VS Code with `code .` from inside the project folder. Your PowerShell prompt should end with `\berealclone>` before any `npx` or `npm` command. VS Code title bar shows the open folder — check it matches.

```
// The check + fix sequence
# Before EVERY session – confirm your location:
$ pwd
# Expected: C:\Users\Win\Desktop\Coding_Stuff\React_Native\berealclone
# If in the wrong folder:
$ cd berealclone
$ pwd # verify again
# Open VS Code from inside the project:
$ code .
# The dot = "open THIS directory", not the parent.
# VS Code title bar should show:
# berealclone [not React_Native]
```

Error #05

Source: app/_layout.tsx + app/(tabs)/_layout.tsx

GestureHandlerRootView missing — crash on gesture**HIGH****What is this error?**

The app crashed or the BottomSheet on the Profile screen failed to respond when the user tried to interact with it. The error referenced gesture handler not being set up.

Type

Runtime crash on first gesture interaction. Silent failure on BottomSheet open.

Root Cause

react-native-gesture-handler requires GestureHandlerRootView to be an ancestor in the React component tree. This sets up the gesture recognition system for everything rendered inside it. BottomSheet from @gorhom/bottom-sheet uses gesture-handler internally. Without the wrapper, gesture events are never set up and the library crashes.

Fix

Wrap your layout with GestureHandlerRootView style={{ flex: 1 }}. It must be a PARENT of gesture components, not a sibling. You added it in both the root _layout.tsx and the (tabs)/_layout.tsx to cover all gesture-based screens.

How to Prevent This Next Time

Any time you install @gorhom/bottom-sheet, react-native-reanimated, or any gesture-based library: check their docs for required wrapper components. GestureHandlerRootView is the most common requirement. Add it once at the root layout and you are covered for the entire app.

Error #06

Source: app/(auth)/login.tsx

SafeAreaView edges prop TypeScript error on Android**MEDIUM****What is this error?**

TypeScript threw a red underline on the edges prop of SafeAreaView. The message: 'Type string[] is not assignable to type SafeAreaViewProps edges'. The tutorial used edges={['top', 'bottom']} and it worked on the instructor's iOS setup.

Type

TypeScript type mismatch. Red underline. SDK version difference between iOS tutorial (SDK 55) and Android setup (SDK 54).

Root Cause

The edges prop on SafeAreaView from react-native-safe-area-context uses stricter TypeScript typing in SDK 54 than in SDK 55. The tutorial instructor was on SDK 55 and iOS — a different combination. The prop value ['top', 'bottom'] that TypeScript accepted there did not match the exact expected type in your setup.

Fix

Remove the edges prop entirely. SafeAreaView still functions correctly as a safe area container without it. The default behaviour handles both top and bottom safe areas on Android.

How to Prevent This Next Time

When TypeScript rejects a prop, hover over it in VS Code to read the exact expected type. Do not guess — read the full error output. When following a tutorial on a different SDK version or platform, expect subtle prop type differences and be ready to adapt.

Error #07

Source: app/(auth)/onboarding.tsx + app.json

Android permission popup never appeared**HIGH****What is this error?**

The image picker opened but showed no images and no permission dialog appeared. On some runs it silently returned status 'denied' without ever asking the user. The permission was declared in app.json but Android ignored it.

Type

Silent runtime failure. No crash. Permission silently denied without Android ever showing the dialog to the user.

Root Cause

app.json permission declarations are native configuration. They must be baked into the Android manifest (AndroidManifest.xml) at build time via `npx expo prebuild`. Hot reload and even `npx expo run:android` do NOT reprocess app.json. The permission was declared in app.json but the APK on the device was built before that declaration existed — so Android had no record of it.

Fix

After any app.json change, run this exact sequence in order: (1) `npx expo prebuild --platform android` (2) `echo sdk.dir=C:\Users\Win\AppData\Local\Android\Sdk > android/local.properties` (3) `npx expo run:android`. Step 2 is critical — prebuild wipes local.properties and the app will silently crash without it.

How to Prevent This Next Time

app.json = native config = baked at build time. Hot reload = JavaScript only. These are two separate worlds. Never confuse them. Any time you add permissions, icons, splash screens, or plugins to app.json: you must prebuild before testing.

```
// The mandatory 3-step sequence after any app.json change
# Step 1: bake app.json into native Android manifest
$ npx expo prebuild --platform android
# WARNING: this deletes android/local.properties
# Step 2: recreate local.properties IMMEDIATELY after prebuild
$ echo sdk.dir=C:\Users\Win\AppData\Local\Android\Sdk > android/local.properties
# Without this: silent crash on launch, no error message
# Step 3: build and install the updated APK
$ npx expo run:android
# Data warning: run:android = up to 300MB. WiFi only.
```

Error #08

Source: Supabase Dashboard

Supabase storage bucket policy mess**HIGH****What is this error?**

Profile image uploads were failing. Reading uploaded images also failed. The Supabase storage bucket had auto-generated policies with wrong names and wrong SQL conditions. The bucket visibility did not match tutorial expectations.

Type

Supabase storage policy configuration error. Upload and read requests rejected with permission errors.

Root Cause

Supabase dashboard templates auto-generate policies with generic SQL conditions. The tutorial expected specific policy names and SQL that the templates did not produce. Additionally the bucket was accidentally set to public when restricted access was needed. Auto-generated policies had names like 'Allow public access' instead of the specific names the tutorial code checked against.

Fix

Delete all auto-generated policies from the profiles bucket. Manually create 4 policies: (1) Public SELECT — all roles, bucket_id = 'profiles'. (2) Authenticated INSERT — authenticated role, folder ownership check. (3) Authenticated UPDATE — authenticated role, SELECT + UPDATE operations. (4) Authenticated DELETE — authenticated role, SELECT + DELETE operations.

How to Prevent This Next Time

Always create Supabase storage policies manually when following a tutorial. Auto-generated templates have generic conditions that rarely match specific tutorial setups. Check the exact SQL in the tutorial and replicate it. Test each policy immediately after creating it with a real upload/read attempt.

Error #09

Source: app/(auth)/onboarding.tsx

Typo: 'username' instead of 'username' in Supabase query**MEDIUM****What is this error?**

The username duplicate check in `handleComplete` never worked. Any username could be registered multiple times. No error was thrown. No warning appeared. The check silently always returned no match.

Type

Silent logic bug. No crash. No TypeScript error. Supabase queried a non-existent column and returned empty results every time.

Root Cause

The Supabase query had `.eq('username', username)` — missing the letter 'r'. Supabase attempted to query a column called 'username' which does not exist in the profiles table. It returned 0 results silently. `existingUser` was always null. TypeScript cannot catch this — it is a string value, not a type. The column name is only checked at runtime against the actual database schema.

Fix

Change `.eq('username', username)` to `.eq('username', username)` in `onboarding.tsx`. Then test the fix by trying to register a duplicate username. You should see the 'Username already taken' alert.

How to Prevent This Next Time

Test every Supabase query path manually after writing it. TypeScript checks types, not string values. Column name typos compile clean but break silently at runtime. Use the Supabase Table Editor SQL panel to verify exact column names before coding queries.

```
// The typo – spot the difference
.eq("username", username) // WRONG – column does not exist
.eq("username", username) // CORRECT – matches profiles table
// What Supabase does with the wrong column name:
// 1. Sends the query to PostgreSQL
// 2. PostgreSQL finds no column called "username"
// 3. Returns 0 rows
// 4. .single() returns null
// 5. if (existingUser) is always false
// 6. Duplicate usernames register freely
// 7. No error. No warning. Complete silence.
```

The 10 Prevention Laws

These are not suggestions. Every single one of these laws was written in blood from a real bug hit during these 2 days. Violate them and you will hit the same walls again.

- 1 Always run `pwd` before starting a coding session**

Your biggest time-waster was VS Code in the wrong folder. Every session: open PowerShell, `cd` into `berealclone`, verify with `pwd`, THEN run code. Your prompt must end with `\berealclone>` before any command.
- 2 Never hardcode auth state**

`const isAuthenticated = false` is never acceptable in real code. Auth state must come from Context via `useAuth()`. Hardcoded values are testing shortcuts that always get committed by accident.
- 3 Read the full error message before touching any code**

Every error tells you: what went wrong, which file, which line, and often the fix. Three times in these sessions you changed code before understanding the error. Read it completely. Diagnose first. Fix second.
- 4 Memorise the `app.json` change sequence**

1. `npx expo prebuild --platform android` 2. Recreate `android/local.properties` 3. `npx expo run:android`. Skipping step 2 = silent crash with no error message. Non-negotiable.
- 5 `useEffect([])` is for setup — never for post-action navigation**

Empty dependency array = run once on mount. Navigation that happens after a button press belongs INSIDE the button handler. If you catch yourself putting `router.push` inside `useEffect([])`, stop and reconsider.
- 6 Never call `useAuth()` in the component that renders `AuthProvider`**

The Provider must be mounted ABOVE the component that reads from it. Two-function pattern: `RootLayout` renders `Provider`, `RootLayoutNav` (inside) calls `useAuth()`. This is non-negotiable — the tree order is physics, not preference.
- 7 Test every Supabase query path manually**

TypeScript checks types, not string values in query conditions. Column name typos compile clean and break silently at runtime. After writing any query, test it with real data. Check exact column names in the Supabase dashboard.

One fix at a time — test between each**8**

Multiple bugs were harder to diagnose because several fixes were applied at once. Fix one thing, save, test, confirm it changed something. If the same error appears after two different fix attempts, stop and completely rethink.

Check data cost before any command that downloads**9**

`npx expo run:android` can use 300MB+. `npx expo prebuild` makes network requests. `npx expo start --clear` is safe (no downloads). Any command with `install` in the name: check data cost first. Your hotspot budget is real.

Canary and alpha packages are broken on Android until proven otherwise**10**

The `@expo/ui` canary package worked on the instructor's iOS but crashed on your Android. Before installing any package, check if the version contains 'canary', 'alpha', or 'nightly'. Check the GitHub Issues page for Android-specific problems before adopting it.

Quick Cheat Sheet

Print this. Stick it on your monitor. Save your sanity.

Essential Commands

What you want	Command	Notes
Start dev server (JS only)	<code>npx expo start</code>	For JS/UI changes only
Clear Metro cache	<code>npx expo start --clear</code>	Safe. No downloads.
Full native rebuild	<code>npx expo run:android</code>	After ANY native package change
Prebuild (after app.json)	<code>npx expo prebuild --platform android</code>	Then recreate local.properties
Recreate local.properties	<code>echo sdk.dir=...Sdk > android/local.properties</code>	After prebuild
Clean Gradle cache	<code>cd android && .\gradlew clean && cd ..</code>	Before rebuilding after native changes
Check current directory	<code>pwd</code>	Run before every session
Enter project folder	<code>cd berealclone</code>	Must be in project root
Check connected devices	<code>adb devices</code>	Before run:android
Open VS Code in project	<code>code .</code>	From inside berealclone folder
Delete node_modules	<code>Remove-Item -Recurse -Force node_modules</code>	Then npm install

Concepts Quick Reference

Concept	One-liner	From your code
useState	Persistent variable + re-render	<code>const [email, setEmail] = useState("")</code>
useEffect([])	Run once on mount	Auth guard, navigation check
useEffect([x])	Run when x changes	Refetch when user.id changes
createContext	Create the data pipe	<code>const AuthContext = createContext(undefined)</code>
AuthProvider	Inject data into pipe	<code><AuthContext.Provider value={{user}}></code>
useAuth()	Read from pipe anywhere	<code>const { signIn } = useAuth()</code>
useRef	Hold ref without re-render	<code>const sheetRef = useRef(null)</code>
useCallback	Memoize a function	<code>const open = useCallback(() => {}, [])</code>
async/await	Wait for Promise cleanly	<code>const { data, error } = await supabase...</code>
try/catch/finally	Handle async errors	finally: always stop the spinner

router.push	Navigate forward (backable)	<code>router.push('/(auth)/signup')</code>
router.replace	Navigate — no going back	<code>router.replace('/(tabs)')</code>
?. chaining	Null-safe access	<code>navigationState?.key</code>
interface	TypeScript object shape	<code>interface AuthContextType { user: User }</code>
User null	Union type	Either a User object or null

You Are Ready For EarGuard.

Every concept in these 3 PDFs maps directly to what EarGuard needs. The patterns are identical. Only the domain changes.

AuthContext → SoundMonitorContext — holds audio state, monitoring on/off, decibel level

signIn / signUp → startMonitoring / stopMonitoring — async functions in your context

useEffect cleanup → Foreground service lifecycle — start on mount, stop on cleanup

useRef for sheet → useRef for audio recorder instance — imperative control, no re-render

isAuth guard → isMonitoring guard — redirect to permissions screen if not granted

Supabase query → Native module call — same async/await + try/catch pattern

The most important skill built in these 2 days is not React Native syntax. It is the debugging mindset: read the full error, identify the file, understand the root cause, apply one fix, verify it changed something. That process works in any framework, any language, any project.

Immediate next steps:

1. Remove the zombie useEffect from signup.tsx (lines 22-24).
2. Fix password minimum from 3 to 6 in signup.tsx.
3. Fix the username typo to username in onboarding.tsx.
4. Implement setUser() in AuthContext after signIn/signup succeeds.
5. Replace const isAuth = false with const { user } = useAuth() in _layout.tsx.
6. Test the full signup to onboarding to home flow end to end.

Built from real pain

Every error in all 3 parts of this guide was hit face-first. The zombie useEffect, the wrong folder, the username typo, the prebuild sequence — all real. All documented. Now go build EarGuard.

Spanish word of the day: el error

error (eh-ROAR) = the error / the bug. Example: Encontre un error en mi codigo. (I found a bug in my code.) You found 9 of them. Nueve errores. You are already fluent in the most important vocabulary.